

Lab 1B

Lab 1b – Operations, Secrets, and Incident Response

Prerequisite: Lab 1a completed and verified

1. Project Overview (What This Lab Is About)

In Lab 1a, you built a working system.

In Lab 1b, you will operate, observe, break, and recover that system.

You will extend your EC2 → RDS application to include:

- Dual secret storage

 - AWS Systems Manager Parameter Store

 - AWS Secrets Manager

- Centralized logging via CloudWatch Logs

- Automated alarms when database connectivity fails

- Incident-response actions using previously saved values

This lab simulates what happens after deployment, which is where most real cloud work lives.

2. Why This Lab Exists (Real-World Context)

Most cloud failures are not caused by:

- Bad code

- Missing features

- Wrong instance sizes

They are caused by:

- Credential issues

- Secret rotation failures

- Misconfigured access

- Silent connectivity loss

- Poor observability

This lab teaches you how to design for failure, detect it early, and recover.

ly, and recover using stored configuration data.

3. Workforce Relevance (Why Employers Care)

This Lab Maps Directly to Job Responsibilities

In the workforce, you will be expected to:

- Know where secrets live

- Understand why multiple secret systems exist

- Diagnose outages using logs and metrics

- Respond to incidents without redeploying everything

- Restore service quickly using known-good configuration

This is the difference between:

- “I can deploy AWS resources”

- and

- “I can keep systems running under pressure”

4. Parameter Store vs Secrets Manager (Conceptual)

You will store database connection values in both systems and intentionally use them during recovery.

AWS Systems Manager Parameter Store

Best for:

- Configuration values

- Non-rotating data

- Shared application parameters

Supports:

- Plain text

- SecureString (encrypted)

Often used for:

- Feature flags

- Endpoints

- Environment configuration

AWS Secrets Manager

Best for:

- Credentials
- Passwords
- Rotating secrets

Supports:

- Automatic rotation
- Tight audit integration

Often used for:

- Database passwords
- API keys

Industry Reality:

Many environments use both – intentionally.

5. What You Are Building in Lab 1b

New Capabilities Added

- 1) Store DB values in Parameter Store
- 2) Store DB credentials in Secrets Manager
- 3) Log application DB connection failures to CloudWatch Logs
- 4) Create a CloudWatch Alarm that triggers when failures exceed a threshold
- 5) Simulate a DB outage or credential failure
- 6) Recover the system using saved parameters/secrets without redeploying EC2

This is operations engineering, not app development.

6. Expected Deliverables (What You Must Produce)

A. Configuration Artifacts

Parameter Store entries for:

- DB endpoint

DB port
DB name

Secrets Manager secret for:
DB username/password

B. Observability

Application logs visible in CloudWatch Logs
Explicit log entries on DB connection failure
CloudWatch Alarm configured on error count

C. Incident Response Proof

Evidence of a simulated failure
Evidence of alarm triggering
Evidence of successful recovery using stored values

7. Technical Verification Using AWS CLI (Required)

You must verify everything via CLI – not screenshots alone. What? You think this is easy?

7.1 Verify Parameter Store Values

```
aws ssm get-parameters \  
  --names /lab/db/endpoint /lab/db/port /lab/db/name \  
  --with-decryption
```

Expected:

Parameter names returned
Correct DB endpoint and port

7.2 Verify Secrets Manager Value

```
aws secretsmanager get-secret-value \  
  --secret-id lab/rds/mysql
```

Expected:

JSON output

Fields:

- username
- password
- host
- port

7.3 Verify EC2 Can Read Both Systems

From EC2:

```
aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
```

Expected:

- Both commands succeed
- No AccessDeniedException

7.4 Verify CloudWatch Log Group Exists

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/ec2/lab-rds-app
```

Expected:

- Log group present

7.5 Verify DB Failure Logs Appear

Simulate failure (examples):

- Stop RDS
- Change DB password in Secrets Manager without updating DB
- Block SG temporarily

Then check logs:

```
aws logs filter-log-events \
  --log-group-name /aws/ec2/lab-rds-app \
```

```
--filter-pattern "ERROR"
```

Expected:

Explicit DB connection failure messages

7.6 Verify CloudWatch Alarm

```
aws cloudwatch describe-alarms \  
  --alarm-name-prefix lab-db-connection
```

Expected:

Alarm present

State transitions to ALARM during failure

7.7 Incident Recovery Verification

After restoring correct credentials or connectivity:

```
curl http://<EC2_PUBLIC_IP>/list
```

Expected:

Application resumes normal operation

No redeployment required

8. Incident-Response Focus (What This Lab Teaches)

During recovery, you must:

Identify failure source via logs

Retrieve correct values from:

Parameter Store

Secrets Manager

Restore service using configuration – not guesswork

This mirrors real on-call workflows.

9. Common Failure Modes (And Why They Matter)

Failure	Real-World Meaning	
-----	-----	

Alarm never fires	Poor observability	
Logs lack detail	Weak incident diagnostics	
EC2 can't read parameters	IAM misdesign	
Recovery requires redeploy	Fragile architecture	

10. What Completing Lab 1b Proves

If you complete this lab, you can confidently say:

"I can operate, monitor, and recover AWS workloads using proper secret management and observability."

That is mid-level engineer capability, not entry-level.

11. Reflection Questions: Answer all of these

- A) Why might Parameter Store still exist alongside Secrets Manager?
- B) What breaks first during secret rotation?
- C) Why should alarms be based on symptoms instead of causes?
- D) How does this lab reduce mean time to recovery (MTTR)?
- E) What would you automate next?

Lab 1B - My files

 [Lab 1B - Complete Walkthrough & Incident Response Guide](#)

Lab 1B — Step-by-step (what you're supposed to do)

0) Prereq check

1. Confirm **Lab 1A is completed and verified** (EC2 → RDS app is working end-to-end).
2. Make sure you have the key values from Lab 1A handy:
 - RDS endpoint (host) - `lab-mysql.XXXXXXXXXX.us-east-2.rds.amazonaws.com`

- Port (likely 3306 for MySQL)
 - DB name - labdb
 - DB username/password
 - EC2 public IP and your app endpoints (example: `/list`)
 - Your EC2 instance role (IAM role) name
-

1) Store DB connection values in SSM Parameter Store

1. Create Parameter Store entries for:

- `/lab/db/endpoint` = your RDS endpoint
- `/lab/db/port` = your DB port
- `/lab/db/name` = your database name

2. Use **SecureString** if you want these encrypted. (Endpoint/name can be plain String if you choose, but SecureString is fine too.)

CLI verification (required):

1. Run:

```
aws ssm get-parameters \  
  --names /lab/db/endpoint /lab/db/port /lab/db/name \  
  --with-decryption
```

1. Confirm the parameters return and values match what you expect.


```
CloudShell

us-east-2 +

~ $ aws ssm get-parameters \
> --names /lab/db/endpoint /lab/db/port /lab/db/name \
> --with-decryption
{
  "Parameters": [
    {
      "Name": "/lab/db/endpoint",
      "Type": "String",
      "Value": "lab-mysql.chkce02amfxr.us-east-2.rds.amazonaws.com",
      "Version": 3,
      "LastModifiedDate": "2026-01-03T23:20:46.390000+00:00",
      "ARN": "arn:aws:ssm:us-east-2:919113286081:parameter/lab/db/endpoint",
      "DataType": "text"
    },
    {
      "Name": "/lab/db/name",
      "Type": "String",
      "Value": "lab-mysql",
      "Version": 1,
      "LastModifiedDate": "2026-01-03T22:54:18.028000+00:00",
      "ARN": "arn:aws:ssm:us-east-2:919113286081:parameter/lab/db/name",
      "DataType": "text"
    },
    {
      "Name": "/lab/db/port",
      "Type": "String",
      "Value": "3306",
      "Version": 1,
      "LastModifiedDate": "2026-01-03T22:52:39.661000+00:00",
      "ARN": "arn:aws:ssm:us-east-2:919113286081:parameter/lab/db/port",
      "DataType": "text"
    }
  ]
}
```

2) Store DB credentials in Secrets Manager

1. Create a secret in Secrets Manager with ID:

- `lab/rds/mysql`

2. Put a JSON payload in the secret that includes:

- `username`
- `password`
- `host`
- `port`

CLI verification (required):

1. Run:

```
aws secretsmanager get-secret-value \
--secret-id lab/rds/mysql
```

1. Confirm the JSON contains the required fields.



```
CloudShell
us-east-2 +
~ $ aws secretsmanager get-secret-value \
> --secret-id lab/rds/mysql
{
  "ARN": "arn:aws:secretsmanager:us-east-2:919113286081:secret:lab/rds/mysql-Ss0Dz3",
  "Name": "lab/rds/mysql",
  "VersionId": "09b0ca09-c175-4709-92fc-7d95be0713b9",
  "SecretString": "{\"username\":\"admin\",\"password\":{\"!0$1>Iu]dN0h601zRGWjH0R1ng7!\",\"engine\":\"mysql\",\"host\":\"lab-mysql.chkce02amfr.us-east-2.rds.amazonaws.com\",\"port\":\"3306\",\"dbInstanceIdentif",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2026-01-01T03:18:10.851000+00:00"
}
```

3) Ensure EC2 can read both systems (IAM + runtime verification)



Current Status:  **Complete** — EC2 can read both SSM Parameter Store and Secrets Manager.



Lesson Learned: The original lab instructions said "update the EC2 instance role policies" but did **not** provide the actual IAM policy JSON or step-by-step console instructions. Figuring out the correct `ssm:GetParameter` and `ssm:GetParameters` actions, scoping them to `arn:aws:ssm:us-east-2:123456789012:parameter/lab/db/*`, and attaching them as an inline policy was **my responsibility to troubleshoot and solve**.

This is intentional. Real-world cloud work doesn't come with copy-paste answers. The lab tests whether you can:

- Read an error message (`AccessDeniedException`)
- Identify the missing permission (`ssm:GetParameter`)
- Construct a least-privilege policy
- Attach it correctly

This gap between "what the instructions say" and "what you actually have to do" is the skill being tested.

Policies needed on role `RDS-Lab-Access` :

1) Secrets Manager (already attached ):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSpecificSecret",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:us-east-2:123456789012:secret:lab/rds/mysql*"
    }
  ]
}
```

2) SSM Parameter Store (attached ):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadLabParameters",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters"
      ],
      "Resource": "arn:aws:ssm:us-east-2:123456789012:parameter:lab/db/*"
    }
  ]
}
```

 **Fix: Add SSM Permission**

1. Go to **IAM Console** → **Roles** → `RDS-lab-Access`
2. Click **Add permissions** → **Create inline policy**
3. Switch to **JSON** tab and paste the SSM policy above
4. Name it `lab-ssm-read`
5. **Save**
6. Retry from EC2:

```
aws ssm get-parameter --name /lab/db/endpoint
```

Troubleshooting Checklist:

- ✓ **Trust Relationship:** Does the IAM Role have ``ec2.amazonaws.com`` as a trusted entity?
 - *Execution:* Check IAM Role > Trust relationships tab for `Service: `ec2.amazonaws.com``.
- ✓ **Exact ARN Match:** Run `aws secretsmanager list-secrets` to copy the exact ARN. Does it match the policy resource?
 - *Execution:* Run the command, copy the ARN, and strictly compare it with the IAM policy Resource field.
- ✓ **Hidden Encryption:** Secrets Manager *always* encrypts. If it used a custom KMS key from Lab 1A by mistake, the role needs `kms:Decrypt`.
 - *Result:* Using default `aws/secretsmanager` key — no extra permissions needed.
- ✓ **VPC Endpoints:** Is there a VPC Endpoint for Secrets Manager blocking access?
 - *Result:* No VPC endpoint for Secrets Manager exists — not a factor.

Detailed Troubleshooting Instructions

▼ ✓ 1. Trust Relationship Check

Goal: Confirm the IAM Role trusts EC2 to assume it.

1. Open the **IAM Console** → **Roles**

2. Find and click on your EC2 instance role (the one attached to your Lab 1 EC2)
3. Click the **Trust relationships** tab
4. Look for this in the trusted entities JSON:

```
"Service": "ec2.amazonaws.com"
```

1. **Pass:** You see `ec2.amazonaws.com` listed → Move on
2. **Fail:** It's missing → Click **Edit trust policy** and add EC2 as a trusted service

▼ 2. Exact ARN Match

Goal: Confirm your policy's Resource ARN matches the actual secret ARN exactly.

1. SSH into your EC2 or use CloudShell
2. Run:

```
aws secretsmanager list-secrets --query "SecretList[?Name='lab/rds/mysql'].ARN" --output text
```

1. Copy the full ARN returned (example: `arn:aws:secretsmanager:us-east-2:919113286081:secret:lab/rds/mysql-AbCdEf`)
2. Go to **IAM Console** → **Roles** → your EC2 role → **Permissions**
3. Find the policy with `secretsmanager:GetSecretValue`
4. Compare the `Resource` field character-by-character with what you copied
5. **Pass:** They match (or your policy uses a valid wildcard like `lab/rds/mysql*`) → Move on
6. **Fail:** Mismatch → Edit the policy to use the correct ARN

▼ 3. Hidden Encryption / KMS Key Check

Goal: Determine if a custom KMS key is encrypting your secret, and if so, grant `kms:Decrypt` .

1. Open **Secrets Manager Console**

2. Click on your secret: `lab/rds/mysql`
3. Scroll to **Encryption key** section
4. Check what key is listed:
 - **If it says `aws/secretsmanager` (the AWS-managed key):** No extra permissions needed → Move on
 - **If it shows a custom key (like one from Lab 1A):** Copy the KMS Key ARN and continue below
5. Go to **IAM Console** → **Roles** → your EC2 role
6. Add this permission to an existing or new inline policy:

```
{
  "Effect": "Allow",
  "Action": "kms:Decrypt",
  "Resource": "<paste-your-kms-key-arn-here>"
}
```

1. Save and retry your `aws secretsmanager get-secret-value` command from EC2

▼ 4. VPC Endpoint Check

Goal: Rule out a VPC Endpoint for Secrets Manager that might be blocking your request.

1. Run this command (from EC2 or CloudShell):

```
aws ec2 describe-vpc-endpoints \
  --filters "Name=service-name,Values=*secretsmanager*" \
  --query "VpcEndpoints[*].{ID:VpcEndpointId,Service:ServiceName,State:State,Policy:PolicyDocument}"
```

1. **If no results returned:** No VPC Endpoint exists → This is not your issue → Move on
2. **If a VPC Endpoint exists:**

- Check the **Policy** field for `Deny` statements or restrictive `Allow` statements
- Check the **Security Group** attached to the endpoint:

```
aws ec2 describe-vpc-endpoints \
  --filters "Name=service-name,Values=*secretsmanager*" \
  --query "VpcEndpoints[*].Groups"
```

- Confirm that Security Group allows inbound HTTPS (port 443) from your EC2's Security Group or subnet
3. **If the endpoint policy or SG is blocking you:** Either fix the endpoint policy/SG, or delete the endpoint so requests go over the public internet instead



Suggested order: Work items 3 and 4 since 1 and 2 are already checked off. Start with **#3 (KMS)** since it's the most common hidden cause of `AccessDenied` on Secrets Manager.

1. Update the **EC2 instance role policies** so the EC2 instance can:
 - Read SSM parameters you created
 - Read the Secrets Manager secret `lab/rds/mysql`
2. SSH into EC2 and verify access **from EC2**:

```
aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
```

1. Expected result: both succeed, and you do *not* see `AccessDeniedException`.
- 2.

```

[ec2-user@ip-10-212-10-161 ~]$ aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
{
  "Parameter": {
    "Name": "/lab/db/endpoint",
    "Type": "String",
    "Value": "lab-mysql.chkce02amfxr.us-east-2.rds.amazonaws.com",
    "Version": 3,
    "LastModifiedDate": "2026-01-03T23:20:46.390000+00:00",
    "ARN": "arn:aws:ssm:us-east-2:919113286081:parameter/lab/db/endpoint",
    "DataType": "text"
  }
}
{
  "ARN": "arn:aws:secretsmanager:us-east-2:919113286081:secret:lab/rds/mysql-SxMDz3",
  "Name": "lab/rds/mysql",
  "VersionId": "09b0ca89-c175-4709-92fc-7d95be0713b9",
  "SecretString": "{\"username\":\"admin\",\"password\":\"i8$l>Iu}dNOh601zzRGfiHJRlmg7\\\", \"engine\":\"mysql\\\", \"host\":\"lab-mysql.chkce02amfxr.us-east-2.rds\"}"
}

```

4) Add centralized logging to CloudWatch Logs

Goal: Get your application's logs (especially DB connection errors) flowing into CloudWatch so you have a central place to diagnose issues.

Cleanup: Start Fresh (Run These First If Redoing Step 4)

If you've already attempted Step 4 and want to start over cleanly:

```

# Stop and remove the CloudWatch Agent
sudo systemctl stop amazon-cloudwatch-agent
sudo yum remove -y amazon-cloudwatch-agent

# Remove any config files you created
sudo rm -rf /opt/aws/amazon-cloudwatch-agent

# Remove the log directory if you created it
sudo rm -rf /var/log/rdsapp

# Undo systemd log redirect changes (if you made them)
# Edit the service file and remove StandardOutput/StandardErr
or lines:
sudo nano /etc/systemd/system/rdsapp.service
# Then reload:
sudo systemctl daemon-reload
sudo systemctl restart rdsapp

```

In AWS Console:

- **CloudWatch** → **Log groups** → Delete `/aws/ec2/lab-rds-app` if it exists
- **CloudWatch** → **Alarms** → Delete `lab-db-connection-failure-alarm` if it exists
- **IAM** → **Roles** → `RDS-Lab-Access` → Remove `CloudWatchAgentServerPolicy` or any inline CloudWatch policy you added

Once cleaned up, proceed below.

Step 4.0 — Redirect your Flask app logs to a file



Your Setup (from Lab 1A):

- **Language:** Python (Flask)
- **App location:** `/opt/rdsapp/ app.py`
- **Process manager:** systemd (service: `rdsapp`)

Because your app runs as a **systemd service**, logs go to **journald** by default — not a file. CloudWatch Agent reads files more easily, so we'll redirect the output to a log file.

1. Create the log directory:

```
sudo mkdir -p /var/log/rdsapp
```

1. Edit the systemd service to redirect output to a file:

```
sudo nano /etc/systemd/system/rdsapp.service
```

1. Add these two lines under the `[Service]` section:

```
StandardOutput=append:/var/log/rdsapp/app.log
StandardError=append:/var/log/rdsapp/app.log
```

Your full `[Service]` section should now look like:

```
[Service]
WorkingDirectory=/opt/rdsapp
Environment=SECRET_ID=lab/rds/mysql
ExecStart=/usr/bin/python3 /opt/rdsapp/app.py
Restart=always
StandardOutput=append:/var/log/rdsapp/app.log
StandardError=append:/var/log/rdsapp/app.log
```

1. Reload and restart the service:

```
sudo systemctl daemon-reload
sudo systemctl restart rdsapp
```

1. Verify logs are flowing to the file:

```
sudo tail -f /var/log/rdsapp/app.log
```

You should see Flask startup messages. Press `Ctrl+C` to exit.

Step 4.0.5 — Add ERROR logging to your Flask app



Critical: Your current `app.py` doesn't explicitly print `ERROR` when DB connections fail. Without this, your CloudWatch alarm won't trigger. You must add explicit error logging.

1. Edit your app:

```
sudo nano /opt/rdsapp/app.py
```

1. Find the `get_conn()` function and replace it with this version:

```
def get_conn():
    try:
```

```

c = get_db_creds()
host = c["host"]
user = c["username"]
password = c["password"]
port = int(c.get("port", 3306))
db = c.get("dbname", "labdb")
return pymysql.connect(host=host, user=user, password=
=password, port=port, database=db, autocommit=True)
except Exception as e:
    print(f"ERROR: Database connection failed - {e}")
    raise

```

1. Restart the app:

```
sudo systemctl restart rdsapp
```

Now when DB connections fail, `ERROR: Database connection failed` will appear in your logs — exactly what CloudWatch will filter on.

Step 4.1 — Install and configure the CloudWatch Agent on EC2

1. SSH into your EC2 instance
2. Install the CloudWatch Agent:

```
sudo yum install -y amazon-cloudwatch-agent
```

(If using Ubuntu/Debian, use `apt` instead)

1. Create a CloudWatch Agent config file:

```
sudo nano /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwa
tch-agent.json
```

1. **Paste this configuration** (this is your exact path based on Lab 1A setup):

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/rdsapp/app.log",
            "log_group_name": "/aws/ec2/lab-rds-app",
            "log_stream_name": "{instance_id}",
            "timezone": "UTC"
          }
        ]
      }
    }
  }
}
```



Log Path Explained: `/var/log/rdsapp/app.log` is the file you configured in Step 4.0 when you added `StandardOutput` to your systemd service. This is where your Flask app now writes all output.

1. Start the CloudWatch Agent:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config \
  -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json \
  -s
```

1. Verify the agent is running:

```
sudo systemctl status amazon-cloudwatch-agent
```

```
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service → /etc/systemd/system/amazon-cloudwatch-agent.service.
ec2-user@ip-10-212-10-161 ~]$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: disabled)
   Active: active (running) since Sun 2026-01-04 01:03:03 UTC; 13s ago
     Main PID: 7420 (amazon-cloudwat)
        Tasks: 6 (limit: 1120)
      Memory: 44.3M
         CPU: 203ms
    CGroup: /system.slice/amazon-cloudwatch-agent.service
            └─7420 /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml -env

Jan 04 01:03:03 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7424]: 2026/01/04 01:03:03 Executing /opt/aws/amazon-cloudwatch-agent
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: D! [EC2] Found active network interface
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: I! imds retry client will retry 1 times! Detected the instanc
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: 2026/01/04 01:03:04 Reading json config file path: /opt/aws/am
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.j
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: 2026/01/04 01:03:04 Reading json config file path: /opt/aws/am
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: 2026/01/04 01:03:04 I! Valid Json input schema.
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: I! Trying to detect region from ec2
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7429]: 2026/01/04 01:03:04 Configuration validation first phase succe
Jan 04 01:03:04 ip-10-212-10-161.us-east-2.compute.internal start-amazon-cloudwatch-agent[7420]: I! Detecting run_as_user...
lines 1-20/20 (END)
```

i-0bac30e743a1d2d4f (lab-ec2-app)

Step 4.2 — Add IAM permissions for CloudWatch Logs

Your EC2 instance role (`RDS-Lab-Access`) needs permission to write logs.

1. Go to **IAM Console** → **Roles** → `RDS-Lab-Access`
2. Click **Add permissions** → **Attach policies**
3. Search for and attach: `CloudWatchAgentServerPolicy`

Or add an inline policy with these minimum permissions:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",
        "logs:DescribeLogStreams",
        "logs:DescribeLogGroups"
      ],
      "Resource": [
```

```

    "arn:aws:logs:us-east-2:123456789012:log-group:/aws/ec2/lab-rds-app:*",
    "arn:aws:logs:us-east-2:123456789012:log-group:*"
  ]
}
]
}

```



Lesson Learned: `DescribeLogStreams` and `DescribeLogGroups` are **two different permissions**:

Permission	What it does
<code>DescribeLogStreams</code>	List streams <i>inside</i> a log group
<code>DescribeLogGroups</code>	List/describe the log groups themselves

Why two resources? `DescribeLogGroups` operates at the log-group level (not stream level), so the specific `/aws/ec2/lab-rds-app:*` resource doesn't cover it. Adding `log-group:*` lets you describe groups.

Step 4.3 — Make sure your app logs DB errors with the word "ERROR"

Your application code needs to explicitly log connection failures. Example patterns:

Node.js example:

```

connection.connect((err) => {
  if (err) {
    console.error('ERROR: Database connection failed -', err.message);
    // The word "ERROR" is critical for CloudWatch filtering
  }
});

```

Python example:

```
try:
    connection = mysql.connector.connect(...)
except mysql.connector.Error as err:
    logging.error(f"ERROR: Database connection failed - {err}")
```



Critical: The word `ERROR` must appear in your log lines. This is what CloudWatch will filter on later. Without it, your alarm won't trigger.

Step 4.4 — CLI verification

Wait 2-3 minutes for logs to start flowing, then run:

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/ec2/lab-rds-app
```

Expected result: You see your log group listed:

```
{
  "logGroups": [
    {
      "logGroupName": "/aws/ec2/lab-rds-app",
      ...
    }
  ]
}
```

```
[ec2-user@ip-10-212-10-161 ~]$ aws logs describe-log-groups \
--log-group-name-prefix /aws/ec2/lab-rds-app
{
  "logGroups": [
    {
      "logGroupName": "/aws/ec2/lab-rds-app",
      "creationTime": 1767490236316,
      "metricFilterCount": 0,
      "arn": "arn:aws:logs:us-east-2:919113286081:log-group:/aws/ec2/lab-rds-app:*",
      "storedBytes": 0,
      "logGroupClass": "STANDARD",
      "logGroupArn": "arn:aws:logs:us-east-2:919113286081:log-group:/aws/ec2/lab-rds-app"
    }
  ]
}
```

Troubleshooting if log group doesn't appear:

- Check agent status: `sudo systemctl status amazon-cloudwatch-agent`
- Check agent logs: `sudo tail -f /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log`
- Verify IAM permissions are attached
- Confirm your app is actually writing to the log file path you specified

5) Create a CloudWatch Alarm for DB connectivity failures

Goal: Set up an alarm that automatically triggers when your app logs too many DB connection errors.

Step 5.1 — Create a Metric Filter (turns log patterns into metrics)

Before you can alarm on errors, you need to convert log entries into a countable metric.

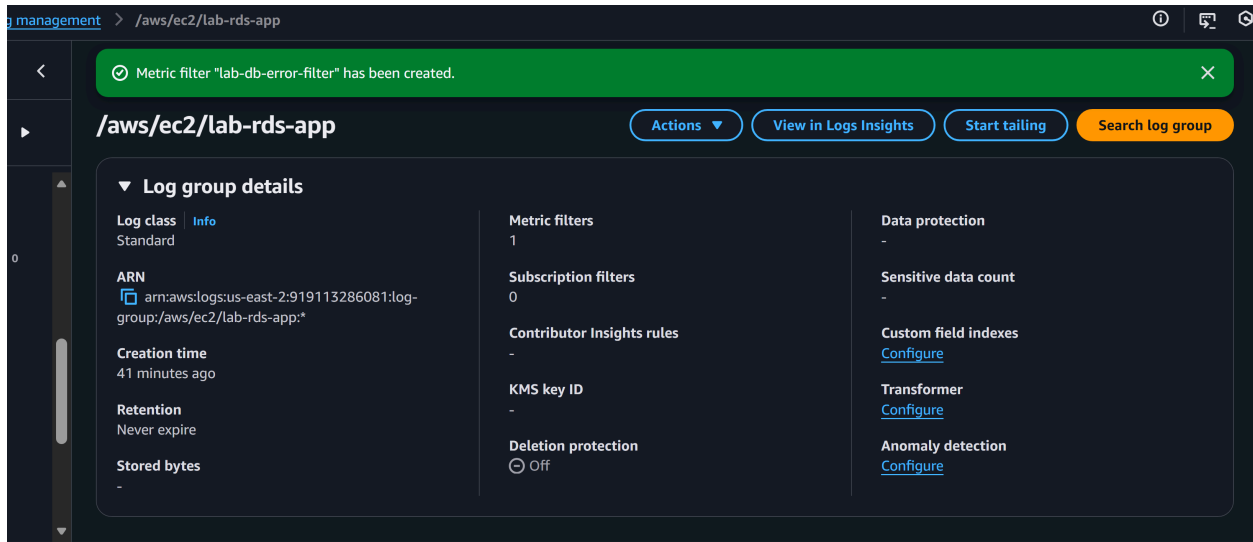
Option A: AWS Console

1. Go to **CloudWatch Console** → **Log groups**
2. Click on `/aws/ec2/lab-rds-app`
3. Click **Metric filters** tab → **Create metric filter**
4. **Filter pattern:** `ERROR`
5. Click **Test pattern** to verify it matches your error logs
6. Click **Next**
7. **Filter name:** `lab-db-error-filter`
8. **Metric namespace:** `Lab/Application`

9. **Metric name:** `DBConnectionErrors`

10. **Metric value:** `1`

11. Click **Create metric filter**



We're going to be able to create the metric if you have an error already logged. So you need to trigger an error in order for you to even make the metric.

Option B: AWS CLI

```
aws logs put-metric-filter \
  --log-group-name /aws/ec2/lab-rds-app \
  --filter-name lab-db-error-filter \
  --filter-pattern "ERROR" \
  --metric-transformations \
    metricName=DBConnectionErrors,metricNamespace=Lab/Application,metricValue=1
```

Step 5.2 — Create the CloudWatch Alarm

Option A: AWS Console

1. Go to **CloudWatch Console** → **Alarms** → **Create alarm**
2. Click **Select metric**

3. Navigate to: **Lab** → **Application** → **DBConnectionErrors**
4. Select the metric and click **Select metric**
5. Configure the alarm:
 - **Statistic:** Sum
 - **Period:** 1 minute
 - **Threshold type:** Static
 - **Whenever DBConnectionErrors is:** Greater than
 - **Threshold value:** `0` (or `2` if you want some tolerance)
6. **Alarm name:** `lab-db-connection-failure-alarm`
7. **Alarm description:** "Triggers when DB connection errors are detected in application logs"
8. **(Optional)** Configure SNS notification if you want email alerts
9. Click **Create alarm**

Option B: AWS CLI

```
aws cloudwatch put-metric-alarm \  
  --alarm-name lab-db-connection-failure-alarm \  
  --alarm-description "Triggers when DB connection errors exc  
eed threshold" \  
  --metric-name DBConnectionErrors \  
  --namespace Lab/Application \  
  --statistic Sum \  
  --period 60 \  
  --threshold 1 \  
  --comparison-operator GreaterThanOrEqualToThreshold \  
  --evaluation-periods 1 \  
  --treat-missing-data notBreaching
```

Step 5.3 — CLI verification

```
aws cloudwatch describe-alarms \  
  --alarm-name-prefix lab-db-connection
```

Expected result: Your alarm appears with state OK (no errors yet):

```
{  
  "MetricAlarms": [  
    {  
      "AlarmName": "lab-db-connection-failure-alarm",  
      "StateValue": "OK",  
      ...  
    }  
  ]  
}
```

6) Simulate an incident (break the system on purpose)

Goal: Intentionally cause a DB connectivity failure to prove your observability stack works.



Pick ONE method below. The Security Group method is the safest and easiest to reverse.

Option A: Block connectivity via Security Group (Recommended)

Why this is best: Quick, reversible, doesn't touch your actual DB or credentials.

1. Find your RDS Security Group:

```
aws rds describe-db-instances \  
  --db-instance-identifier lab-mysql \  
  --query "DBInstances[0].VpcSecurityGroups[*].VpcSecurityGro
```

```
upId" \  
  --output text
```

1. **Find the inbound rule that allows your EC2 to connect** (port 3306):

```
aws ec2 describe-security-groups \  
  --group-ids sg-01f5bc3e6b016dbb2 \  
  --query "SecurityGroups[0].IpPermissions"
```

1. **Remove (revoke) the inbound rule temporarily:**

```
aws ec2 revoke-security-group-ingress \  
  --group-id sg-XXXXXXXXXXXXXXXXXX \  
  --protocol tcp \  
  --port 3306 \  
  --source-group sg-XXXXXXXXXXXXXXXXXX \
```

1. **Trigger your app** to attempt a DB connection:

```
curl http://<EC2_PUBLIC_IP>/list
```

1. **You should see a connection timeout or error.** Your app should now be logging `ERROR` lines.



Save this command to restore later:

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-XXXXXXXXXXXXXXXXXX \  
  --protocol tcp \  
  --port 3306 \  
  --source-group sg-XXXXXXXXXXXXXXXXXX \
```

```

    "RevokedSecurityGroupRules": [
      {
        "SecurityGroupRuleId": "sgr-037b650adaebae915",
        "GroupId": "sg-01f5bc3e6b016dbb2",
        "IsEgress": false,
        "IpProtocol": "tcp",
        "FromPort": 3306,
        "ToPort": 3306,
        "ReferencedGroupId": "sg-012119988bf5e29f1"
      }
    ]
  }
}
~ $ curl http://3.129.73.57/list
<!doctype html>
<html lang=en>
<title>500 Internal Server Error</title>
<h1>Internal Server Error</h1>
<p>The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.</p>
~ $

```

Untitled

7) Prove observability (logs + alarm actually work)

Goal: Verify that your incident was detected by both logs and the alarm.

Step 7.1 — Check CloudWatch Logs for ERROR entries

```

aws logs filter-log-events \
  --log-group-name /aws/ec2/lab-rds-app \
  --filter-pattern "ERROR" \
  --limit 10

```

Expected result: You see log entries with explicit DB connection failure messages:

```

{
  "events": [
    {
      "message": "ERROR: Database connection failed - Connect
ion timed out",
      "timestamp": 1735955000000,
      ...
    }
  ]
}

```

If you see no events:

- Confirm your app is actually logging (check the local log file on EC2)
- Confirm the CloudWatch Agent is running
- Confirm the log file path in your agent config matches reality
- Wait another minute and retry (there's a small delay)

Step 7.2 — Check the CloudWatch Alarm state

```
aws cloudwatch describe-alarms \  
  --alarm-names lab-db-connection-failure-alarm \  
  --query "MetricAlarms[0].StateValue"
```

Expected result: `"ALARM"`

You can also check alarm history:

```
aws cloudwatch describe-alarm-history \  
  --alarm-name lab-db-connection-failure-alarm \  
  --history-item-type StateUpdate \  
  --max-records 5
```

Expected: You see a transition from `OK` → `ALARM`



Success criteria:

- ✓ ~~ERROR logs appear in CloudWatch Logs~~
- ✓ ~~Alarm state shows `ALARM`~~
- ✓ ~~Alarm history shows the state transition~~

```
{
  "MetricAlarms": [
    {
      "AlarmName": "lab-db-connection-failure-alarm",
      "AlarmArn": "arn:aws:cloudwatch:us-east-2:919113286081:alarm:lab-db-connection-failure-alarm",
      "AlarmDescription": "triggers when DB connection errors are detected in application logs",
      "AlarmConfigurationUpdatedTimestamp": "2026-01-04T02:27:23.521000+00:00",
      "ActionsEnabled": true,
      "OKActions": [],
      "AlarmActions": [
        {
          "arn:aws:sns:us-east-2:919113286081:labdb-Cloudwatch-Alarm"
        }
      ],
      "InsufficientDataActions": [],
      "StateValue": "ALARM",
      "StateReason": "Threshold Crossed: 1 out of the last 1 datapoints [1.0 (04/01/26 02:13:00)] was greater than the threshold (0.0) (minimum 1 datapoint for OK -> ALARM transition).",
    }
  ]
}
```

8) Incident response + recovery (no redeploy)

Goal: Restore the system to working order using your stored configuration—without redeploying EC2.

Step 8.1 — Use logs to identify the failure source

Run this to see recent errors:

```
aws logs filter-log-events \
  --log-group-name /aws/ec2/lab-rds-app \
  --filter-pattern "ERROR" \
  --start-time $(date -d '10 minutes ago' +%s)000 \
  --limit 20
```

Read the error messages. They should tell you:

- "Connection timed out" → Network/SG issue
- "Access denied" → Credential mismatch
- "Unknown host" → Endpoint/DNS issue

Step 8.2 — Retrieve known-good values from Parameter Store & Secrets Manager

Get your stored DB connection values:

```
aws ssm get-parameters \
  --names /lab/db/endpoint /lab/db/port /lab/db/name \
  --with-decryption \
  --query "Parameters[*].{Name:Name,Value:Value}"
```

Get your stored credentials:

```
aws secretsmanager get-secret-value \
  --secret-id lab/rds/mysql \
  --query "SecretString" \
  --output text | jq .
```

Step 8.3 — Fix the root cause (based on which method you used)

If you used Security Group blocking (Option A):

```
aws ec2 authorize-security-group-ingress \
  --group-id <your-rds-sg-id> \
  --protocol tcp \
  --port 3306 \
  --source-group <your-ec2-sg-id>
```

If you stopped RDS (Option B):

```
aws rds start-db-instance --db-instance-identifier lab-mysql
```

Wait for status to become "available":

```
aws rds wait db-instance-available --db-instance-identifier lab-mysql
```

If you changed the password (Option C):

Update Secrets Manager back to the correct password:

```
aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"<CORRECT_PASSWORD>","host":"lab-mysql.XXXXXXXXXX.us-east-2.rds.amazonaws.com","port":"3306"}'
```


Then restart your app to pick up the corrected secret.

Step 8.4 — Verify recovery WITHOUT redeploying

```
curl http://<EC2_PUBLIC_IP>/list
```

Expected result: Your app returns data successfully.

Also verify the alarm clears:

```
aws cloudwatch describe-alarms \
  --alarm-names lab-db-connection-failure-alarm \
  --query "MetricAlarms[0].StateValue"
```

Expected: `"OK"` (may take 1-2 minutes to transition)



Incident resolved! You just:

1. Detected an outage via logs
2. Were alerted via CloudWatch Alarm
3. Retrieved stored configuration
4. Fixed the issue
5. Restored service without redeploying

This is real incident response.

```
~ $ aws cloudwatch describe-alarms \
>   --alarm-names lab-db-connection-failure-alarm \
>   --query "MetricAlarms[0].StateValue"
"ALARM"
~ $
```

9) Deliverables you must produce (make sure you can show proof)

1. Configuration artifacts exist:
 - Parameter Store: endpoint, port, name
 - Secrets Manager: username/password/host/port
 2. Observability proof:
 - Log group exists
 - DB failure logs captured with explicit ERROR lines
 - Alarm triggers during the incident
 3. Incident response proof:
 - Evidence of simulated failure
 - Evidence of alarm triggering
 - Evidence of successful recovery using stored values
-

10) Reflection questions (must answer all)

1. Answer:
 - A) Why might Parameter Store still exist alongside Secrets Manager?

My top of mind thought would be because the secrets manager just stores the password to access the thing, and the parameter store holds the finer details of the individual infrastructure that you're building. You may change out the parameters of something like your RDS information (as in the resource number could change). And if it did change, you will want to have that track so that you know what infrastructure you're working on as you're scaling up.



Review: You're on the right track — you correctly identified the core distinction (credentials vs. configuration). Here's what you could add:

- **Cost:** Parameter Store standard parameters are **free**. Secrets Manager charges ~\$0.40/secret/month. For non-sensitive config, Parameter Store saves money.
- **Rotation:** Secrets Manager has **built-in automatic rotation** with Lambda integration. Parameter Store doesn't.
- **History:** Parameter Store existed first (2016); Secrets Manager came later (2018) specifically for credentials. Many orgs already had Parameter Store in place.
- **Access patterns:** Parameter Store is optimized for high-throughput reads of config values; Secrets Manager is optimized for secure credential storage with audit trails.

Bottom line: Use both intentionally — Parameter Store for config, Secrets Manager for credentials.

- B) What breaks first during secret rotation?

My best guess to this question is that the passwords themselves break, and that it won't rotate properly, and you have to go in and reconfigure the passwords. That's what I think first.



Review: You're circling the issue but the real answer is about **timing and synchronization**. Here's what actually breaks:

1. Secrets Manager rotates the password (updates the secret)
2. A Lambda function updates the password **on the database side**
3. But your **application still has the OLD password cached in memory**
4. App tries to connect → "Access denied" → Outage

What breaks first: The **application layer** — because it's holding stale credentials while the DB and Secrets Manager have already moved on.

The fix: Applications must re-fetch credentials on connection failure (or use short TTL caching). This is why you saw the "restart your app" step in Option C recovery — the app needed to pick up the new credentials.

- C) Why should alarms be based on symptoms instead of causes?

When I say this shit, alarms are based on symptoms - things that are happening. It allows us to be more reactive to the infrastructure because it's similar. I would think of it like a doctor. A cause is something you would need to diagnose; it's a diagnosis, and you can't go off of a cause based on the thing because that doesn't get to the root of the problem. You have to have an understanding of what is happening (the symptoms) and then the programmer can analyze that data and identify the cause and prescribe the right remedy. There is a difference in medicine - the remedy could be healing or a poison. Having a cause removes that ability to pick the right remedy. We need and we may put in a poison instead.



Review: The doctor analogy is solid and shows good intuition. Here's the technical precision to add:

- **Symptom-based alarm:** "DB connections are failing" → Catches ANY cause (SG change, credential issue, DB down, network partition, DNS failure, etc.)
- **Cause-based alarm:** "Security Group was modified" → Only catches ONE specific cause, misses everything else

Why symptoms win:

1. You can't predict every failure mode
2. A single symptom alarm catches multiple root causes
3. Detection comes first, diagnosis comes second
4. Users don't care WHY it's broken — they care that it IS broken

Your lab example: Your alarm fired on **ERROR** logs (a symptom). It would have fired whether the cause was SG blocking, RDS stopped, or bad credentials.

- D) How does this lab reduce mean time to recovery (MTTR)?

And the way this lab reduces mean time to recovery. It gives you a comprehensive understanding and puts you through a scenario where you can experiment with failures within the system. Then you have to solve and figure out that issue without having to redeploy the infrastructure because redeploying it takes a long time and also costs money. Whereas you fixing it within keeps the system going and your updating it.



Review: You're correct about "no redeploy = faster recovery." Here's the full breakdown of how each component reduces MTTR:

Component	How it reduces MTTR
CloudWatch Alarm	Faster detection — you know immediately, not after user complaints
CloudWatch Logs	Faster diagnosis — error messages tell you what's wrong
Parameter Store	Faster recovery — known-good config is pre-stored, no guessing
Secrets Manager	Faster recovery — correct credentials are retrievable instantly
This lab page	Faster response — documented runbook means no improvisation

MTTR = Detection + Diagnosis + Recovery. This lab optimized all three.

- E) What would you automate next?

Right now, I had put all of this and did it in ClickOps and in the cloud shell, but I would instead make a Terraform configuration out of this so that I can instantly deploy the infrastructure every single time. Then, anytime I needed to make changes, I would only have to adjust the code instead of having to do ClickOps.



Review: Terraform is a strong answer — IaC is the logical next step. Other valid answers you could mention in an interview:

- **Auto-remediation:** Lambda triggered by the CloudWatch Alarm that automatically restores the SG rule (self-healing infrastructure)
- **Automated secret rotation:** Secrets Manager can rotate credentials on a schedule with zero human intervention
- **CI/CD pipeline:** Auto-deploy app changes with rollback on failure
- **Systems Manager Automation:** Runbooks that execute recovery steps automatically

Your answer shows you understand the ClickOps → IaC progression, which is exactly what employers want to hear.



[Command Line & Linux Basics Explained](#)



[Lab 1B Terminology Glossary](#)